Práctica 2: GPIO - Entradas y salidas digitales

Belmont Muñoz Samuel
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 317318289

Domínguez Palacios Jesús Alejandro
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 421066205

Dorantes Mejía Emmanuel
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 313004766

García Olvera Emiliano
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 314256009

Hernández Alonso Antonio Adonahí
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 316339825

Jiménez Reyes Alan Yael
Ingeniería en Computación
Facultad de Ingeniería
No. Cuenta: 320026726

***Resumen*—Este reporte presenta el desarrollo de la Práctica 2 sobre el manejo de entradas y salidas de propósito general (GPIO) en la Raspberry Pi. Se exploró el comportamiento eléctrico de los pines, las diferencias de numeración y las arquitecturas entre generaciones de Raspberry Pi. Mediante la librería libgpiod, se controlaron actuadores (LEDs) y se leyeron sensores (botones) utilizando lenguaje C. Finalmente, se analizó la diferencia de rendimiento del procesador al leer entradas mediante polling frente a la lectura basada en eventos, culminando con el diseño de un semáforo peatonal.**

I. Objetivo

Los objetivos principales de esta práctica son:

- Controlar pines GPIO como salida (LEDs) y entrada (botones) desde C con `libgpiod`.
- Comprobar con hardware real la diferencia entre leer una entrada por polling y por eventos.
- Entender qué es y cómo se comporta eléctricamente un pin GPIO.
- Armar un circuito básico en protoboard.

II. Introducción

Los pines GPIO (General Purpose Input/Output) son conexiones cuyo comportamiento no está fijo de fábrica. El software decide, en tiempo de ejecución, si actúan como entrada (leyendo voltaje) o como salida (forzando un voltaje).

Al trabajar con la Raspberry Pi, es fundamental comprender sus limitaciones eléctricas para no dañar el hardware:

- Los pines operan estrictamente a niveles lógicos de 3.3V y no son tolerantes a 5V.
- Tienen un límite de corriente de 16 mA por pin.
- Tienen un límite de 50 mA sumando todos los pines activos.

Por estas limitaciones, es obligatorio el uso de resistencias (ej. 220Ω) al conectar LEDs.

Otro aspecto crucial es la numeración de los pines. Existen dos sistemas: el físico (o BOARD), que numera los pines según su posición en el header de 40 pines, y el BCM, que es el número real en el chip Broadcom. En esta práctica se utiliza la numeración BCM, ya que es la que espera la librería `libgpiod`.

Además, es importante notar la evolución del hardware. En las versiones anteriores (como la RPi 3 o 4), el GPIO estaba integrado directamente en el procesador (SoC) como un periférico "memory-mapped". En la Raspberry Pi 5, el control del GPIO se movió a un chip externo llamado RP1. Por esta razón, se utiliza `libgpiod` en lugar de librerías antiguas, asegurando que el código funcione de manera transparente en cualquier versión de la placa.

III. Desarrollo

III-A. Material y Equipo

- Raspberry Pi (con SSH y conexión a red configurados).
- 4 LEDs (Rojo, Amarillo, Verde, Azul).
- 4 Resistencias de 220Ω para los LEDs.
- 2 Botones pulsadores.
- 2 Resistencias de 10kΩ para configuración pull-down.
- Protoboard y cables puente (jumpers).

III-B. Procedimiento

III-B1. Armado del circuito (Parte A): Se armó un circuito base utilizando la numeración BCM. Se conectaron los cuatro LEDs a los pines GPIO 17, 27, 22 y 23. Cada LED se protegió conectando su ánodo al pin GPIO a través de una resistencia de 220Ω, y el cátodo a tierra (GND). Para las entradas, se conectaron dos botones a los pines GPIO 5 y 6. Se utilizó una configuración de resistencia pull-down (10kΩ conectada a GND) para garantizar que el pin lea un 0 lógico en reposo, y un 1 lógico al presionar el botón (conectado a 3.3V).

III-B2. Exploración desde la terminal (Parte B): Antes de programar en C, se interactuó con los pines directamente desde la terminal de comandos para verificar su estado en el sistema operativo. Primero, se empleó `gpioinfo` para listar y conocer el controlador asignado (en este caso, `gpiochip0`). Posteriormente, se probó forzar los valores lógicos desde la terminal usando `gpioset -c gpiochip0 -p 3s 17=1` para encender el primer LED temporalmente, y se monitorearon las entradas con `gpioget` verificando el cambio de estado (0 a 1) al presionar el botón físico.



Figura 1: Conexiones BCM en el header de la Raspberry Pi.

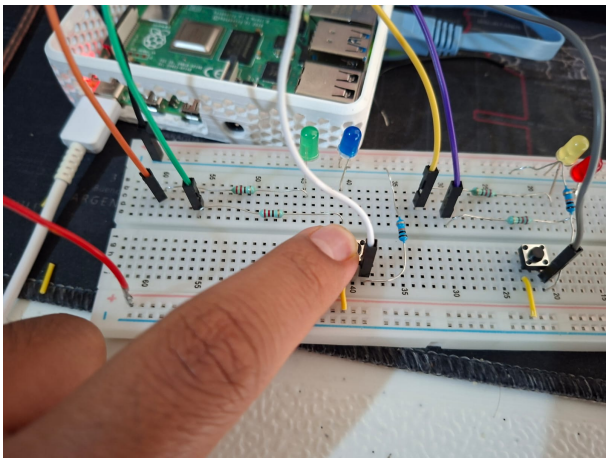


Figura 2: Circuito físico en el protoboard mostrando los LEDs y configuración pull-down.

```
konan@konan: ~
konan@konan:~ $ gpioinfo
gpiochip0 - 58 lines:
        line   0:       "ID_SDA"                input
        line   1:       "ID_SCL"                input
        line   2:       "GPIO2"                 input
        line   3:       "GPIO3"                 input
        line   4:       "GPIO4"                 input
        line   5:       "GPIO5"                 input
        line   6:       "GPIO6"                 input
        line   7:       "GPIO7"                 input
        line   8:       "GPIO8"                 input
        line   9:       "GPIO9"                 input
        line  10:       "GPIO10"                input
        line  11:       "GPIO11"                input
        line  12:       "GPIO12"                input
        line  13:       "GPIO13"                input
        line  14:       "GPIO14"                input
        line  15:       "GPIO15"                input
        line  16:       "GPIO16"                input
        line  17:       "GPIO17"                output
        line  18:       "GPIO18"                input
        line  19:       "GPIO19"                input
        line  20:       "GPIO20"                input
        line  21:       "GPIO21"                input
        line  22:       "GPIO22"                input
        line  23:       "GPIO23"                input
        line  24:       "GPIO24"                input
        line  25:       "GPIO25"                input
        line  26:       "GPIO26"                input
        line  27:       "GPIO27"                input
        line  28:       "RGMII_MDIO"            input
        line  29:       "RGMIO_MDC"             input
        line  30:       "CTS0"                  input
        line  31:       "RTS0"                  input
        line  32:       "TXD0"                  input
        line  33:       "RXD0"                  input
        line  34:       "SD1_CLK"               input
        line  35:       "SD1_CMD"               input
        line  36:       "SD1_DATA0"             input
        line  37:       "SD1_DATA1"             input
        line  38:       "SD1_DATA2"             input
```

Figura 3: Listado de líneas y configuración con el comando gpioinfo.

```
konan@konan: ~
konan@konan:~ $ gpioset -c gpiochip0 -p 3s 17=1
gpioset -c gpiochip0 17=0
```

Figura 4: Uso de gpioset para manipular directamente el valor de la salida digital.

III-B3. Control de LEDs desde C (Parte C): Se procedió a automatizar el hardware mediante código. Se creó el archivo `leds.c` incluyendo las cabeceras de `libgpiod`. Mediante las funciones de la librería, se solicitaron los pines como líneas de salida. Se desarrolló un bucle anidado que activa cada LED de forma iterativa y los apaga empleando `usleep` para marcar el ritmo visual. Para ejecutar el programa, fue estrictamente necesario enlazar la librería compilando con el flag `-lgpiod`.

```
konan@konan: ~
 GNU nano 8.4                                leds.c *
#include <gpiod.h>
#include <stdio.h>
#include <unistd.h>

#define CHIP "/dev/gpiochip0"
#define N_LEDS 4
#define RONDAS 3

int main(void) {
    static const unsigned int pines[N_LEDS] = {17, 27, 22, 23};

    struct gpiod_chip *chip = gpiod_chip_open(CHIP);
    if (!chip) { perror("gpiod_chip_open"); return 1; }

    struct gpiod_line_settings *ajustes = gpiod_line_settings_new();
    gpiod_line_settings_set_direction(ajustes, GPIOD_LINE_DIRECTION_OUTPUT);
    gpiod_line_settings_set_output_value(ajustes, GPIOD_LINE_VALUE_INACTIVE>

    struct gpiod_line_config *cfg_lineas = gpiod_line_config_new();
    gpiod_line_config_add_line_settings(cfg_lineas, pines, N_LEDS, ajustes);

    struct gpiod_request_config *cfg_peticion = gpiod_request_config_new();
    gpiod_request_config_set_consumer(cfg_peticion, "leds");

    struct gpiod_line_request *peticion =
        gpiod_chip_request_lines(chip, cfg_peticion, cfg_lineas);
    if (!peticion) { perror("gpiod_chip_request_lines"); return 1; }

    for (int ronda = 0; ronda < RONDAS; ronda++) {
        for (int i = 0; i < N_LEDS; i++) {
            gpiod_line_request_set_value(peticion, pines[i], GPIOD_LINE_VAL>
            usleep(200000);
            gpiod_line_request_set_value(peticion, pines[i], GPIOD_LINE_VAL>
        }
        for (int i = N_LEDS - 2; i > 0; i--) {
            gpiod_line_request_set_value(peticion, pines[i], GPIOD_LINE_VAL>
            usleep(200000);

^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify
```

Figura 5: Código fuente de leds.c en el editor nano mostrando las peticiones al gpiochip0.

III-B4. Botón por Polling vs Eventos (Partes D y E): Esta fase consistió en analizar el consumo de recursos computacionales. Primero, se codificó `boton_polling.c`, el cual evalúa el estado del botón GPIO 5 en un bucle `while(1)` infinito de forma síncrona. Al ejecutar la utilidad `top`, se comprobó que este enfoque monopoliza un hilo de procesamiento, elevando el uso de CPU a casi un 100 %.

Para contrastar, se utilizó la herramienta `gpiomon`, que implementa una lectura asíncrona basada en eventos de hardware. El monitor de recursos mostró una reducción drástica en la carga del procesador (cerca de 0 % en espera), confirmando que delegar la detección al kernel mediante interrupciones es mucho más eficiente.

III-B5. Reto Final: Semáforo peatonal (Parte F): El reto requirió integrar múltiples pines y lógica de control de tiempo.

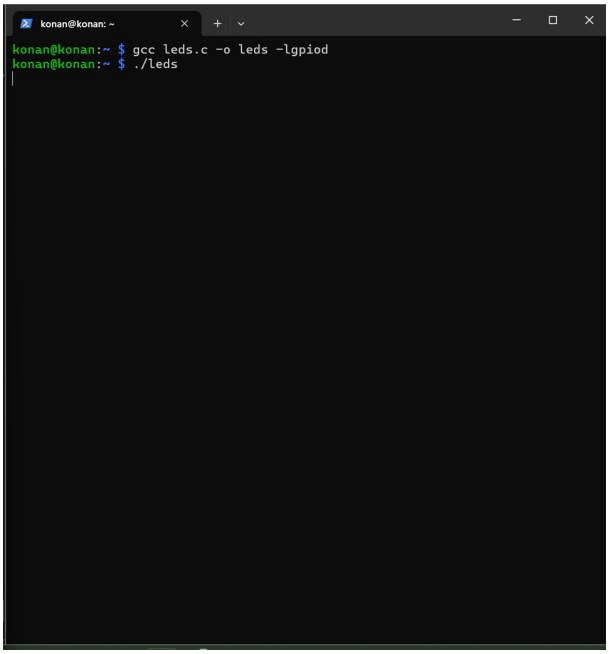


Figura 6: Compilación y ejecución exitosa del programa leds.c.

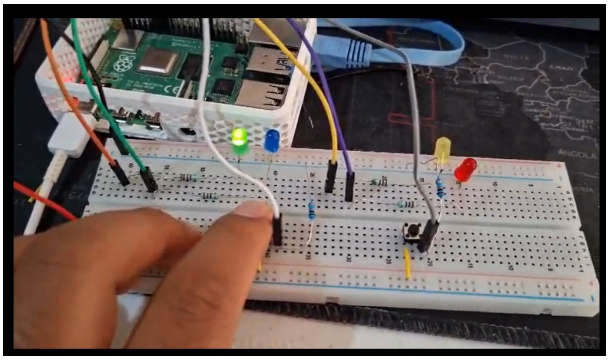


Figura 7: Interacción física con el botón durante las pruebas de lectura de entradas.

Se diseñó un ciclo de estados simulando un semáforo de vehículos (Verde 5s, Amarillo 2s, Rojo 5s). Como se documenta visualmente en la ejecución del circuito, en condiciones normales el sistema alterna automáticamente las luces principales.

Justificación de diseño de software: Dado el análisis previo, para la petición de paso peatonal se implementó la lectura por eventos. Esto permite al semáforo cumplir con los tiempos precisos (`usleep`) de su ciclo sin desperdiciar recursos procesando constantemente el estado del botón. Cuando la interrupción del GPIO 5 se dispara al presionar el botón físico, el sistema termina su fase actual y entra de forma segura al estado rojo, encendiendo simultáneamente el cuarto LED (azul) para indicar al peatón que el cruce es seguro.

IV. Resultados y Análisis

A partir de la ejecución de las pruebas, se observó que la correcta identificación de la numeración BCM y el uso

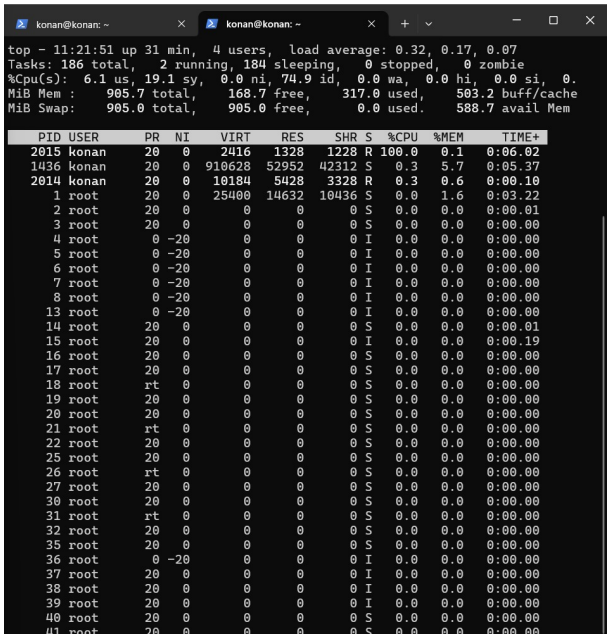


Figura 8: Reporte de `top` demostrando un alto consumo de CPU debido al ciclo de polling.

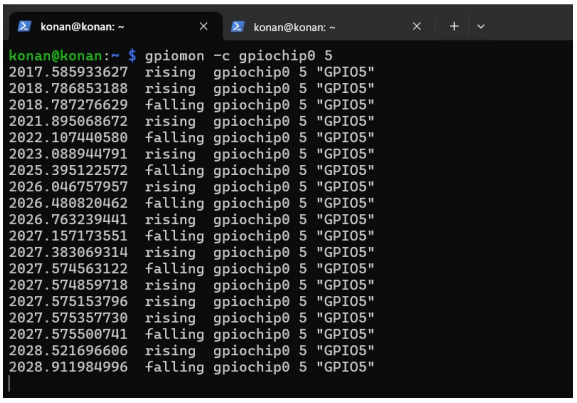


Figura 9: Monitorización de estado mediante `gpiomon`, reaccionando solo en los flancos de subida y bajada.

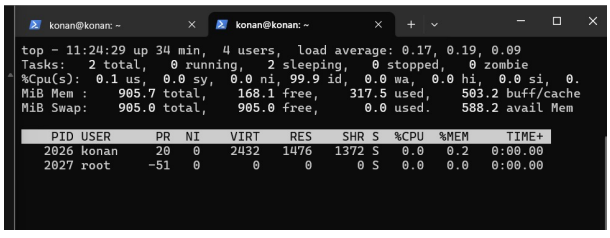


Figura 10: Uso eficiente del CPU (0.0 %) al implementar lecturas orientadas a eventos.

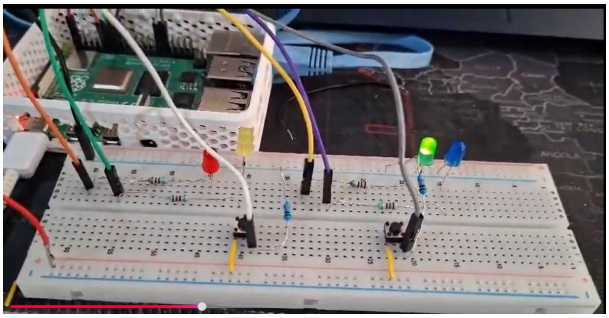


Figura 11: Funcionamiento normal del ciclo automático del semáforo (estado verde activo).

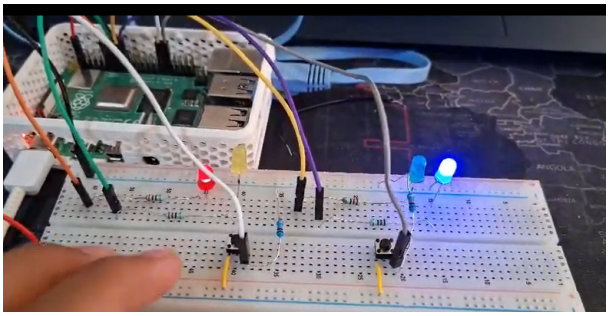


Figura 12: Interrupción por petición peatonal. El botón fuerza el estado seguro (rojo) y activa el LED indicador de cruce (azul).

de resistencias pull-down fueron fundamentales para evitar lecturas ”flotantesz daños en la placa. El análisis más destacable se dio en la comparativa de lectura de entradas. El método de polling (encuestar el pin constantemente en un bucle infinito) demostró ser sumamente ineficiente, acaparando un alto porcentaje de procesamiento del CPU. Por el contrario, el enfoque basado en eventos (como el usado por `gpiomon`) mantuvo el CPU prácticamente al 0 % mientras esperaba. Esto demuestra que, en sistemas embebidos, el diseño de software debe contemplar la gestión de energía y recursos de procesamiento, inclinándose fuertemente por arquitecturas basadas en interrupciones o eventos. La implementación del semáforo comprobó la viabilidad de este último método al integrar entradas interactivas con salidas temporizadas sin bloquear el sistema.

```
konan@konan:~/fse $ git commit -m "Practica 2: GPIO" -m "Belmont Muñoz Samue
l
Domínguez Palacios Jesús Alejandro
Dorantes Mejía Emmanuel
García Olvera Emiliano
Hernández Alonso Antonio Adonahí
Jiménez Reyes Alan Yael"
git log --oneline
git log -1
[master 3db3dd9] Practica 2: GPIO
 4 files changed, 246 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 boton_polling.c
 create mode 100644 leds.c
 create mode 100644 semaforo.c
3db3dd9 (HEAD -> master) Practica 2: GPIO
c891cf9 Practica 1
commit 3db3dd95e4bad04e13bd184a206320aeb72fa6d3 (HEAD -> master)
Author: J Alejandro Domínguez <adp3993@gmail.com>
Date:   Mon Sep 14 11:39:27 2026 -0600

    Practica 2: GPIO

    Belmont Muñoz Samuel
    Domínguez Palacios Jesús Alejandro
    Dorantes Mejía Emmanuel
    García Olvera Emiliano
    Hernández Alonso Antonio Adonahí
    Jiménez Reyes Alan Yael
konan@konan:~/fse $
```

Figura 13: Registro final del repositorio en Git con los entregables de la sesión (`git log --oneline`).

V. Conclusiones

Belmont Muñoz Samuel: [Espacio para conclusión]

Domínguez Palacios Jesús Alejandro: [Espacio para conclusión]

Dorantes Mejía Emmanuel: [Espacio para conclusión]

García Olvera Emiliano: [Espacio para conclusión]

Hernández Alonso Antonio Adonahí: [Espacio para conclusión]

Jiménez Reyes Alan Yael: Durante esta práctica pude aplicar el control a bajo nivel del hardware mediante la Raspberry Pi utilizando la librería libgpiod. Comprendí la vital importancia de no sobrepasar el límite de 3.3V y 16mA por pin para salvaguardar la integridad de la placa. Lo más interesante fue observar en tiempo real, gracias a la herramienta top, cómo un mal diseño de software basado en polling puede desperdiciar recursos del procesador, y cómo el enfoque por eventos soluciona este problema eficientemente. Finalmente, implementar el semáforo me ayudó a consolidar la lógica de programación integrando múltiples salidas y entradas al mismo tiempo.

Referencias

Referencias

[1] ”FSE - Práctica 2: GPIO a fondo”, Documentación del curso Fundamentos de Sistemas Embebidos, UNAM FI.